Construction of SPF Datasets for the ME3AI Project

Abstract

This document specifies the procedure for constructing Survey of Professional Forecasters (SPF) datasets used in this project. The SPF is a quarterly survey of macroeconomic forecasts conducted by the Federal Reserve Bank of Philadelphia; data are provided as Excel workbooks (median and mean levels and growth rates, probabilities, dispersion, and microdata). We describe how to obtain the official releases, which variables and vintages to use, how to validate and normalize the data into the project's cleaned third normal form (3NF) structure, and how to link them to the pipeline input and output folders. This document serves as the single source of truth for SPF dataset construction and reproducibility.

Table 1: Notation and terms used in this document.

Symbol or term	Definition
SPF	Survey of Professional Forecasters
3NF	Third normal form (database normalization)
BEA	Bureau of Economic Analysis
NIPA	National income and product accounts (BEA advance report)
`var_path`	URL path segment for a variable (lowercase; e.g., `ngdp`, `cpi10`)
VAR	Placeholder for variable code in filenames (e.g., NGDP, CPI10)
`overwrite`	Policy: `overwrite` or `skip-if-exists`

1 Data source and scope

The Survey of Professional Forecasters (SPF) is a quarterly survey of professional forecasters, formerly run by the American Statistical Association and the National Bureau of Economic Research (from 1968:Q4) and taken over by the Federal Reserve Bank of Philadelphia in 1990:Q2. The Philadelphia Fed publishes documentation and data files; the canonical reference for variable definitions, file layout, and timing is the *SPF Documentation* (Philadelphia Fed, latest update March 2024), held in this project as `reference/spf-documentation.pdf`.

The survey is timed to the release of the Bureau of Economic Analysis (BEA) advance report of the national income and product accounts (NIPA): questionnaires are sent after that report; deadlines fall in the second to third week of the middle month of each quarter; results are released before the BEA second report. For the construction of our datasets, we use the survey date (year and quarter) as the primary time index. The first survey fully conducted in real time by the Philadelphia Fed is 1990:Q3. Earlier vintages (1968:Q4 to 1990:Q2) may be used with the caveats noted in the official documentation.

2 Published files and variables

The Philadelphia Fed provides SPF data *by variable*: each variable has a dedicated data page from which Excel (and other) files can be downloaded. The variable index is at `https://www.philadelphiafed.org/surveys-and-data/data-files`; clicking a variable (e.g., NGDP, CPI10, RGDP, UNEMP) leads to a page at

`https://www.philadelphiafed.org/surveys-and-data/`*var_path*,

where *var_path* is the variable identifier in lowercase (e.g., `ngdp`, `cpi10`, `rgdp`). The sole exception is CPI inflation, which uses `cpi-spf`. Variable names and paths are listed on the data-files index (e.g., "10-Year CPI Inflation Rate (CPI10)" maps to `/cpi10`).

On each variable page, the following file types are typically offered (availability may vary by variable):

- **Documentation**:
  Link to the SPF documentation PDF (same for all variables).
- **Measures of Cross-Sectional Forecast Dispersion**:
  `Dispersion_`*VAR*`.xlsx` (e.g., `Dispersion_CPI10.xlsx`, `Dispersion_NGDP.xlsx`).
- **Median Responses**:
  `Median_`*VAR*`_Level.xlsx` (e.g., `Median_NGDP_Level.xlsx`, `Median_CPI10_Level.xlsx`).
- **Mean Responses**:
  `Mean_`*VAR*`_Level.xlsx`.
- **Annualized Percent Change of Median/Mean Responses**:
  `Median_`*VAR*`_Growth.xlsx` and `Mean_`*VAR*`_Growth.xlsx`. For variables that have growth rates (e.g., NGDP, RGDP); not all variables such as CPI10 offer growth files.
- **Individual Responses**:
  `Individual_`*VAR*`.xlsx`.

Filenames use the variable code as on the website (e.g., NGDP, CPI10). The automation must fetch the variable page, extract the download links for the desired file types, and save each file to `data-pipeline/input/` under that same filename (no timestamps). Variable names, horizons, and caveats are defined in the SPF documentation; we record the exact variable names and definitions in a project codebook.

3 Download automation: inputs and outputs

We automate retrieval of SPF files from the Philadelphia Fed website so that raw files are saved to `data-pipeline/input/` without manual download. Given a variable name (e.g., CPI10, NGDP), the data are hosted on that variable's page at `https://www.philadelphiafed.org/surveys-and-data/`*var_path*. The automation fetches the page, parses the HTML for the desired download links (Dispersion, Median Level, Mean Level, Median Growth, Mean Growth, Individual), and downloads each file to the input folder under the same filename as on the site (e.g., `Median_CPI10_Level.xlsx`). When parsing links, the implementation must match `<a>` tags where `href` may appear after other attributes (e.g. `class`), since the Philadelphia Fed markup may use that order.

Inputs. A list of variable names (e.g., CPI10, RGDP, UNEMP, NGDP) as on the Philadelphia Fed data-files index. Optional inputs: which file types to download (e.g., median level, mean level, dispersion, individual); overwrite policy `overwrite` $\in$ {`overwrite`, `skip-if-exists`} (whether to overwrite an existing file or skip); output directory (default `data-pipeline/input/`).

Outputs. Each selected file is saved under `data-pipeline/input/` with the same filename as on the website (no timestamps or version suffixes). Each file is downloaded at most once per variable; if multiple variables were to share the same filename, the implementation should deduplicate so that the file is written once.

Three logical functions suffice: (1) given a variable name, return the variable page URL (lowercase path, with exception CPI $\rightarrow$ `cpi-spf`); (2) given a variable page URL and requested file types, fetch the page and return a list of (filename, download URL) pairs; (3) given a download URL and local path, download the file subject to the overwrite policy. The top-level procedure loops over the requested variables, obtains the link list for each, and downloads each file to `data-pipeline/input/`.

4 Pipeline and 3NF

Our data pipeline (see `docs/workflow/data_pipeline.md`) has three stages: **input** (raw data), **cleaned** (third normal form, 3NF), and **output** (analysis results). SPF data enter as raw files in the input folder and are transformed into cleaned tables that satisfy 3NF.

Input. Place the official Excel workbooks in the pipeline input folder. Do not modify the original files; preserve them as the source of truth.

Cleaned. The current implementation focuses on **individual (microdata)** forecasts only. Cleaned output is CSV: `forecast_individual.csv` in **wide** form (primary key: survey_year, survey_quarter, forecaster_id; one column per forecast horizon, all horizons in the same row) and `forecaster_survey.csv` (primary key: survey_year, survey_quarter, forecaster_id; `industry`). No separate survey table is used; the set of (survey_year, survey_quarter) is derivable from `forecast_individual`. Level/mean/median and probability tables may be added in a later stage.

Output. Analyses read from the cleaned folder and write results to the output folder, organized by analysis type.

Before treating a constructed SPF dataset as final: verify that raw files match the expected Philadelphia Fed release; confirm variable names and column layouts align with the SPF documentation; after transformation, check row counts and primary keys; record data source, retrieval date, and any filtering in the codebook or a README in the cleaned folder.

5 CPI10 and realized inflation (CPI1)

Following Appendix A.1.3 in Aruoba, let $\pi_{a\rightarrow b}$ denote annualized quarterly inflation between quarters a and b, and use the same continuous-compounding approximation. The SPF 10-year CPI forecast is a *fixed-event* forecast: in each survey quarter it refers to inflation over 10 years starting from the previous year's Q4. Hence part of the reported object is already realized by the survey date and must be removed before using it as a forward-looking measurement equation.

In the cleaned wide table `forecast_individual.csv`, both the raw 10-year expectation and the realized short-horizon inflation term are taken from the same survey row: column `CPI10` gives the raw 10-year expectation and column `CPI1` gives the realized inflation term used in the adjustment. The adjusted output is written as a separate minimal table `adjusted_cpi10.csv` with columns `(survey_year, survey_quarter, forecaster_id, adjusted_cpi10)`.

With quarterly inflation, the raw 10-year object is the average of 40 quarterly inflation rates. For the first-quarter survey, there is no realized quarter inside the 10-year window, so

$$\text{SPF-10YR-Q1}_t \approx \frac{1}{40}\sum_{j=0}^{39}\pi_{t-1+j\to t+j}. \tag{1}$$

Define adjusted 10-year forecasts by survey quarter as

$$x_{17,t} \equiv \text{CPI10-Q1}_t, \tag{2}$$

$$x_{18,t} \equiv \frac{40\cdot\text{CPI10-Q2}_t - \text{CPI1-Q2}_t}{39}, \tag{3}$$

$$x_{19,t} \equiv \frac{40\cdot\text{CPI10-Q3}_t - (\text{CPI1-Q2}_t + \text{CPI1-Q3}_t)}{38}, \tag{4}$$

$$x_{20,t} \equiv \frac{40\cdot\text{CPI10-Q4}_t - (\text{CPI1-Q2}_t + \text{CPI1-Q3}_t + \text{CPI1-Q4}_t)}{37}. \tag{5}$$

The corresponding forward-looking measurement equations are

$$x_{17,t} = \frac{1}{40}\sum_{j=0}^{39}\pi_{t-1+j\to t+j} + \varepsilon_{17,t}, \tag{6}$$

$$x_{18,t} = \frac{1}{39}\sum_{j=0}^{38}\pi_{t-1+j\to t+j} + \varepsilon_{18,t}, \tag{7}$$

$$x_{19,t} = \frac{1}{38}\sum_{j=0}^{37}\pi_{t-1+j\to t+j} + \varepsilon_{19,t}, \tag{8}$$

$$x_{20,t} = \frac{1}{37}\sum_{j=0}^{36}\pi_{t-1+j\to t+j} + \varepsilon_{20,t}. \tag{9}$$

Under this quarterly representation, CPI1 enters the raw 10-year forecast with weight 1/40 once it is realized, but the adjusted object re-averages only over the remaining forward-looking quarters. Thus Q2 rescales by 39, Q3 rescales by 38, and Q4 rescales by 37. The raw CPI10 measure therefore mechanically contains more realized inflation in Q4 than in Q1.

6 Inflation news

For each forecaster and survey quarter, inflation news is defined as the realized inflation for the current quarter minus the nowcast of that same quarter made in the previous survey. In the cleaned wide table `forecast_individual.csv`, the realized term is `CPI1` from the current survey quarter, while the nowcast term is `CPI2` from the previous survey quarter for the same `forecaster_id`. Thus, for forecaster i in quarter (y, q),

$$\text{news}_{i,y,q} \equiv \text{CPI1}_{i,y,q} - \text{CPI2}_{i,y,q-1}, \qquad q \in \{2,3,4\}, \tag{10}$$

$$\text{news}_{i,y,1} \equiv \text{CPI1}_{i,y,1} - \text{CPI2}_{i,y-1,4}. \tag{11}$$

Equivalently, inflation news at (y, q) is formed by matching the current row (y, q, i) to the lagged survey row $(y, q-1, i)$ when $q \in \{2, 3, 4\}$, and to $(y-1, 4, i)$ when $q = 1$. This construction respects SPF timing: `CPI2` in survey $(y, q-1)$ is the nowcast for inflation realized in quarter (y, q), so the difference isolates the surprise in the realized current-quarter inflation relative to the forecast available one survey earlier.

7 Algorithms: download automation

Algorithm 1 VariablePageURL(*var*)

Require *var*: variable name as on Philadelphia Fed data-files index (e.g., NGDP, CPI10).
path ← lowercase(*var*).
if *var* = `CPI` **then**
    path ← `cpi-spf`. ▷ exception per Philadelphia Fed URL
return `https://www.philadelphiafed.org/surveys-and-data/` ⊕ *path*.

Algorithm 2 GetDownloadLinks(*page_url*, *file_types*)

Require *file_types*: subset of {`dispersion`, `median_level`, `mean_level`, `median_growth`, `mean_growth`, `individual`}, and optionally `documentation`.
Fetch HTML at *page_url*.
Parse page for download links: match `<a>` tags allowing `href` to appear after other attributes (e.g. `class`); match link text or href to each requested type (e.g., "Median Responses" → `Median_VAR_Level.xlsx`, "Measures of Cross-Sectional Forecast Dispersion" → `Dispersion_VAR.xlsx`).
Build list L of (*filename*, *download_url*) for each matched link; extract *filename* from the URL or link (stable name, e.g., `Median_NGDP_Level.xlsx`).
return L.

Algorithm 3 DownloadOneFile(*download_url*, *outpath*, `overwrite`)

Require `overwrite` ∈ {`overwrite`, `skip-if-exists`}.
if `overwrite` = `skip-if-exists` and file exists at *outpath* **then**
    return (skip).
Download from *download_url* to *outpath* (overwrite if file exists when `overwrite` = `overwrite`).
return *outpath*.

Algorithm 4 DownloadByVariableNames(*variable_names*, *file_types*, *out_dir*, `overwrite`)

Require *out_dir* = `data-pipeline/input/` (fixed by project layout).
Require *file_types*: which file types to download per variable (e.g., median level, mean level, dispersion, individual).
written ← ∅. ▷ optional: track filenames already written to avoid duplicate writes
for each v in *variable_names* **do**
  page_url ← VariablePageURL(v).
  L ← GetDownloadLinks(*page_url*, *file_types*).
  for each (*filename*, *url*) in L **do**
    outpath ← *out_dir* joined with *filename*.
    DownloadOneFile(*url*, *outpath*, `overwrite`).
    Add *outpath* to *written* (if not already present).
return *written* (list of paths in `data-pipeline/input/`).

8 Algorithm: construct cleaned datasets (individual only)

The following procedure summarizes the construction of SPF *individual* forecast datasets from raw Excel files to cleaned CSV. Raw files `Individual_VAR.xlsx` are assumed present in `data-pipeline/input/` (e.g., produced by DownloadByVariableNames with `individual` in `file_types`). Output is CSV; `forecast_individual` is written in wide form (one row per survey/forecaster, one column per horizon). Helper functions used by the main procedure are specified below.

Algorithm 5 LoadIndividualSheet(*path*)

Require *path*: path to one `Individual_VAR.xlsx` file.
Load first sheet: row 1 = header; first four columns YEAR, QUARTER, ID, INDUSTRY; remaining columns = horizon names.
Validate: at least YEAR, QUARTER, ID present; at least one horizon column.
Melt to long form: id_vars = (YEAR, QUARTER, ID, INDUSTRY), value_vars = horizon columns; create columns `horizon`, `value`.
Rename YEAR → survey_year, QUARTER → survey_quarter, ID → forecaster_id.
return long-form table (survey_year, survey_quarter, forecaster_id, industry if present, horizon, value).

Algorithm 6 HorizonSortKey(*col*)

Require *col*: horizon column name (e.g., CPI1, CPI10).
Ensure 10-year horizon (name ending with `10`) is ordered last.
return pair (*col* ends with `10`, *col*) for sorting (False before True, then by name).

Algorithm 7 BuildForecastIndividual(*df_long*)

Require *df_long*: concatenated long-form table from LoadIndividualSheet.
Pivot: index = (survey_year, survey_quarter, forecaster_id); columns = horizon; values = value.
If the same horizon appears in more than one raw file, keep the first non-missing value.
Order horizon columns: sort by HorizonSortKey so 10-year column is last.
Final columns: (survey_year, survey_quarter, forecaster_id, horizon_1, horizon_2, ..., horizon_10).
return wide table (key columns as integers where applicable).

Algorithm 8 BuildForecasterSurvey(*df_long*)

Require *df_long*: concatenated long-form table (must contain survey_year, survey_quarter, forecaster_id, INDUSTRY or industry).
Select distinct (survey_year, survey_quarter, forecaster_id, industry); sort by (survey_year, survey_quarter, forecaster_id).
return forecaster_survey table; primary key (survey_year, survey_quarter, forecaster_id).

Algorithm 9 CleanIndividualTo3NF(*input_dir*, *cleaned_dir*)

Require *input_dir*: path to input folder; *cleaned_dir*: path to cleaned output folder.
Find all `Individual_*.xlsx` in *input_dir*.
for each file *path* in that list **do**
    df ← LoadIndividualSheet(*path*).
    Append *df* to collection of long-form tables.
df_long ← concatenate all long-form tables.
forecast_individual ← BuildForecastIndividual(*df_long*).
forecaster_survey ← BuildForecasterSurvey(*df_long*).
Write *forecast_individual* to *cleaned_dir*/`forecast_individual.csv`, *forecaster_survey* to *cleaned_dir*/`forecaster_survey.csv`.
return the two tables (forecast_individual, forecaster_survey).

Algorithm 10 GetQuarterSpecificValue(*key*, *horizon*)

Require *key*:
    (*forecast_individual*, *survey_year*, *survey_quarter*, *forecaster_id*).
Require *forecast_individual*: cleaned wide table.
Find row with key
    (*survey_year*, *survey_quarter*, *forecaster_id*).
return value in column *horizon* (e.g., `CPI1` or `CPI10`).

Algorithm 11 AdjustCPI10Forecasts(*forecast_individual*)

```
Require forecast_individual: cleaned wide table
    containing columns CPI1 and CPI10.
Initialize output table for adjusted 10-year CPI forecasts.
for each row key (survey_year, survey_quarter, forecaster_id) do
    key ← (forecast_individual, survey_year, survey_quarter, forecaster_id)
    cpi10 ← GetQuarterSpecificValue(key, CPI10)
    if survey_quarter = 1 then
        adjusted ← cpi10
    else if survey_quarter = 2 then
        key_q2 ← (forecast_individual, survey_year, 2, forecaster_id)
        cpi1_q2 ← GetQuarterSpecificValue(key_q2, CPI1)
        adjusted ← (40 · cpi10 − cpi1_q2)/39
    else if survey_quarter = 3 then
        key_q2 ← (forecast_individual, survey_year, 2, forecaster_id)
        key_q3 ← (forecast_individual, survey_year, 3, forecaster_id)
        cpi1_q2 ← GetQuarterSpecificValue(key_q2, CPI1)
        cpi1_q3 ← GetQuarterSpecificValue(key_q3, CPI1)
        adjusted ← (40 · cpi10 − (cpi1_q2 + cpi1_q3))/38
    else
        key_q2 ← (forecast_individual, survey_year, 2, forecaster_id)
        key_q3 ← (forecast_individual, survey_year, 3, forecaster_id)
        key_q4 ← (forecast_individual, survey_year, 4, forecaster_id)
        cpi1_q2 ← GetQuarterSpecificValue(key_q2, CPI1)
        cpi1_q3 ← GetQuarterSpecificValue(key_q3, CPI1)
        cpi1_q4 ← GetQuarterSpecificValue(key_q4, CPI1)
        adjusted ← (40 · cpi10 − (cpi1_q2 + cpi1_q3 + cpi1_q4))/37
    Write adjusted to the output table for that row key.
return table of adjusted 10-year CPI forecasts.
```

Algorithm 12 ConstructInflationNews(*forecast_individual*)

```
Require forecast_individual: cleaned wide table
    containing columns CPI1 and CPI2.
Initialize output table for inflation news.
for each row key (survey_year, survey_quarter, forecaster_id) do
    current_key ← (forecast_individual, survey_year, survey_quarter, forecaster_id)
    cpi1_current ← GetQuarterSpecificValue(current_key, CPI1)
    if survey_quarter = 1 then
        lagged_key ← (forecast_individual, survey_year − 1, 4, forecaster_id)
    else
        lagged_key ← (forecast_individual, survey_year, survey_quarter − 1, forecaster_id)
    cpi2_lagged ← GetQuarterSpecificValue(lagged_key, CPI2)
    inflation_news ← cpi1_current − cpi2_lagged
    Write inflation_news to the output table for that row key.
return table of inflation news.
```

Algorithm 13 SummarizeAdjustedCPI10(*adjusted_cpi10*)

Require *adjusted_cpi10*: table with columns (*survey_year*, *survey_quarter*, *forecaster_id*, *adjusted_cpi10*).
Keep rows with non-missing `adjusted_cpi10`.
Compute availability window: first survey, last survey, number of survey quarters with data, and number of non-missing rows.
Count distinct forecasters by (*survey_year*, *survey_quarter*).
For each survey quarter, count distinct forecasters that also have non-missing `adjusted_cpi10` in the previous survey quarter.
return availability summary, forecaster counts by survey, and revision-ready forecaster counts by survey.

9 Notes

Obtaining official Excel files may be implemented by DownloadByVariableNames: given variable names (e.g., CPI10, NGDP), the procedure uses VariablePageURL to form each variable's data page URL (e.g., `https://www.philadelphiafed.org/surveys-and-data/cpi10`), GetDownloadLinks to parse the page for Individual (and optionally other) links, and DownloadOneFile to save each file to `data-pipeline/input/` under the same filename as on the site. The **current** cleaned schema is `forecast_individual`, `forecaster_survey`, and `adjusted_cpi10` for individual (microdata) only; output format is CSV. Level/mean/median and probability tables are not yet in scope; table names and keys should match the pipeline and codebook when added.